

UD5 - Prácticas: alta disponibilidad

Diseño, despliegue y validación de servicios redundantes en entornos de pruebas virtualizados.

Datos de las prácticas	Información
Módulo	Seguridad y Alta Disponibilidad
Curso	2.º ASIR
Modalidad	Semipresencial
Duración estimada	8 horas
Entorno	Máquinas virtuales y redes de entorno de pruebas propias

1. Objetivos

- Identificar SPOF y definir objetivos de disponibilidad, RPO y RTO.
- Desplegar WordPress en dos nodos web con una base de datos independiente.
- Configurar HAProxy con comprobaciones de salud y balanceo de carga.
- Configurar una IP virtual con VRRP y Keepalived en una red aislada.
- Interpretar y validar recursos, quórum y *fencing* en Pacemaker y Corosync.
- Diseñar una plataforma virtualizada disponible con Proxmox y almacenamiento distribuido.

2. Alcance y preparación

Las prácticas se ejecutan exclusivamente en máquinas virtuales, redes internas y direcciones IP del entorno de pruebas. Antes de empezar, crea una *snapshot* de cada VM. No uses IP virtuales, rutas, reglas de NAT, configuraciones de clúster ni credenciales en redes ajenas o de producción.

El itinerario principal requiere cuatro VM AlmaLinux 9 conectadas a una red interna y con acceso temporal a Internet para instalar paquetes:

VM	Hostname	Recursos mínimos	Función
Balanceador	balancer01	1 vCPU, 1-2 GiB RAM	HAProxy.
Web 1	web01	1 vCPU, 2 GiB RAM	Apache, PHP y WordPress.
Web 2	web02	1 vCPU, 2 GiB RAM	Apache, PHP y WordPress.
Base de datos	db01	1 vCPU, 2 GiB RAM	MariaDB.

Asigna IP estáticas propias de tu red. En los ejemplos se usan **IP_BALANCER**, **IP_WEB1**, **IP_WEB2**, **IP_DB** e **IP_VIRTUAL**; sustitúyelas por valores reales. Añade las correspondencias de nombres e IP en **/etc/hosts** de las cuatro máquinas.

```
IP_BALANCER balancer01
IP_WEB1      web01
```

IP_WEB2	web02
IP_DB	db01

3. Práctica 1 - Diagnóstico de disponibilidad

1. Dibuja las dependencias de una web con un único router, firewall, switch, servidor web, base de datos y almacenamiento.
2. Identifica al menos cinco SPOF y propón para cada uno una medida proporcionada: segundo enlace, fuente redundante, SAI, RAID, réplica, backup, balanceador o monitorización.
3. Calcula la disponibilidad con $MTBF = 8,760$ horas y $MTTR = 4$ horas:

$$D = \frac{MTBF}{MTBF + MTTR}$$

4. Calcula el tiempo anual de indisponibilidad de 99.9%, 99.99% y 99.999%. Indica qué medidas reducen MTTR y cuáles aumentan MTBF.
5. Define un RPO y un RTO realistas para la web y justifica la decisión.

4. Práctica 2 - WordPress balanceado con HAProxy

4.1. Preparar las máquinas

En todas las VM, actualiza el sistema, activa el firewall y configura el nombre de host correspondiente:

```
sudo dnf update -y
sudo dnf install -y vim curl wget firewalld
sudo systemctl enable --now firewalld
sudo hostnamectl set-hostname NOMBRE_DE_LA_VM
```

Comprueba conectividad entre las cuatro máquinas mediante `ping`, resolución de nombres mediante `getent hosts web01` y el estado de las interfaces mediante `ip -br a`.

4.2. Configurar MariaDB en db01

Instala y habilita MariaDB:

```
sudo dnf install -y mariadb-server
sudo systemctl enable --now mariadb
sudo firewall-cmd --permanent --add-service=mysql
sudo firewall-cmd --reload
```

Ejecuta `sudo mysql_secure_installation` y aplica las medidas solicitadas. Después, crea una base de datos y un usuario exclusivos del entorno de pruebas. Sustituye `CLAVE_PRUEBAS` por una contraseña distinta de cualquier cuenta personal:

```
sudo mysql
CREATE DATABASE wordpress CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'wpuser'@'IP_WEB1' IDENTIFIED BY 'CLAVE_PRUEBAS';
CREATE USER 'wpuser'@'IP_WEB2' IDENTIFIED BY 'CLAVE_PRUEBAS';
GRANT ALL PRIVILEGES ON wordpress.* TO 'wpuser'@'IP_WEB1';
GRANT ALL PRIVILEGES ON wordpress.* TO 'wpuser'@'IP_WEB2';
FLUSH PRIVILEGES;
EXIT;
```

Comprueba que MariaDB escucha solo en la dirección interna necesaria. No habilites el acceso remoto de `root` ni expongas el puerto MySQL fuera de la red del entorno de pruebas.

4.3. Configurar `web01` y `web02`

En ambos nodos instala Apache y PHP:

```
sudo dnf install -y httpd php php-mysqlnd php-fpm php-gd php-xml php-mbstring tar
sudo systemctl enable --now httpd
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --reload
```

Descarga WordPress en ambos nodos y concede la propiedad al usuario de Apache:

```
cd /tmp
wget https://wordpress.org/latest.tar.gz
tar xzf latest.tar.gz
sudo rm -rf /var/www/html/*
sudo cp -a wordpress/. /var/www/html/
sudo chown -R apache:apache /var/www/html
sudo cp /var/www/html/wp-config-sample.php /var/www/html/wp-config.php
```

Edita `wp-config.php` en cada nodo. Configura `DB_NAME`, `DB_USER`, `DB_PASSWORD` y `DB_HOST` con la información de `db01`. Completa la instalación inicial accediendo primero a uno de los nodos directamente.

Los dos nodos comparten la base de datos, pero las subidas y cambios de archivos siguen siendo un SPOF funcional si no se usa almacenamiento compartido o sincronización. No copies secretos ni archivos de producción en este entorno de pruebas.

4.4. Configurar HAProxy en `balancer01`

Instala HAProxy y permite HTTP:

```
sudo dnf install -y haproxy
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --reload
```

Guarda una copia de `/etc/haproxy/haproxy.cfg` y añade al final una configuración mínima. Sustituye las IP de ejemplo:

```
frontend wordpress_http
    bind *:80
    default_backend wordpress_nodes

backend wordpress_nodes
    balance roundrobin
    option httpchk GET /
    server web01 IP_WEB1:80 check
    server web02 IP_WEB2:80 check

listen stats
    bind *:8080
    stats enable
    stats uri /stats
    stats refresh 10s
```

Valida y activa el servicio:

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
sudo systemctl enable --now haproxy
curl -I http://localhost/
```

Accede a `http://IP_BALANCER/`, revisa el panel `http://IP_BALANCER:8080/stats` desde la red de entorno de pruebas y documenta qué nodos aparecen activos. Detén temporalmente `httpd` en un backend, comprueba que HAProxy lo retira y después restaura el servicio. No confundas balanceo con HA completa: el balanceador y la base de datos siguen siendo dependencias únicas.

5. Práctica 3 - Gateway HA con VRRP y Keepalived

Esta práctica requiere dos gateway Linux y una máquina cliente en una red interna aislada. Cada gateway necesita una interfaz WAN con acceso a Internet y otra interfaz LAN. Ejemplo de LAN:

Equipo	IP LAN
gateway1	192.168.100.2/24
gateway2	192.168.100.3/24
IP virtual VRRP	192.168.100.1/24

Equipo	IP LAN
Cliente	192.168.100.10/24, gateway 192.168.100.1

5.1. Activar enrutamiento y NAT

En ambos gateways, habilita temporalmente el reenvío IP y después persístelo en un fichero de `/etc/sysctl.d/`:

```
sudo sysctl -w net.ipv4.ip_forward=1
echo 'net.ipv4.ip_forward = 1' | sudo tee /etc/sysctl.d/99-ha-routing.conf
sudo sysctl --system
```

Asigna la interfaz WAN a la zona `public` y la LAN a `internal`, adaptando los nombres reales. Habilita NAT únicamente hacia la WAN:

```
sudo firewall-cmd --permanent --zone=public --change-interface=INTERFAZ_WAN
sudo firewall-cmd --permanent --zone=internal --change-interface=INTERFAZ_LAN
sudo firewall-cmd --permanent --zone=public --add-masquerade
sudo firewall-cmd --permanent --zone=internal --set-target=ACCEPT
sudo firewall-cmd --reload
```

Desde el cliente, prueba primero la salida usando la IP real de cada gateway. Verifica las rutas con `ip route` y, si está disponible, `traceroute 8.8.8.8`.

5.2. Configurar Keepalived

Instala Keepalived en ambos gateways:

```
sudo dnf install -y keepalived
```

En `gateway1`, crea `/etc/keepalived/keepalived.conf`. Adapta la interfaz LAN y utiliza una clave de entorno de pruebas:

```
vrrp_instance VI_1 {
    state MASTER
    interface INTERFAZ_LAN
    virtual_router_id 51
    priority 120
    advert_int 1
    authentication {
        auth_type PASS
        auth_pass CLAVE_LAB
    }
    virtual_ipaddress {
```

```

    192.168.100.1/24
  }
}

```

En `gateway2`, usa el mismo `virtual_router_id`, interfaz y dirección virtual, pero configura `state BACKUP` y `priority 100`. Inicia el servicio en ambos nodos:

```

sudo systemctl enable --now keepalived
ip -br address

```

Configura el cliente para usar `192.168.100.1` como gateway. Mantén un `ping 8.8.8.8` activo desde el cliente, desconecta únicamente la interfaz LAN del gateway principal en la configuración de la VM y verifica que la IP virtual aparece en el respaldo. Restaura la interfaz y anota el tiempo de conmutación y los paquetes perdidos.

7. Práctica 5 Evaluable - Máquinas virtuales con Proxmox

Esta práctica necesita tres nodos Proxmox dedicados. La virtualización anidada puede ser lenta y no sustituye la validación en hardware compatible.

1. Instala tres nodos Proxmox con IP estáticas, conectividad mutua y un segundo disco libre en cada nodo para Ceph. Cada nodo debe contar con al menos 2 GiB de RAM; aumenta los recursos cuando el equipo anfitrión lo permita.
2. Crea el clúster desde el primer nodo mediante la interfaz web en `Datacenter > Cluster > Create Cluster` y une los demás con `Join Cluster`, o utiliza `pvecm create NOMBRE_CLUSTER` y `pvecm add IP_NODO_PRINCIPAL`.
3. Verifica el estado con `pvecm status`. Explica por qué tres nodos facilitan mantener quórum frente a una caída.
4. Instala Ceph en los nodos desde `Datacenter > Ceph`, crea un OSD con el disco secundario de cada nodo y un `pool` para las VM. Comprueba que el estado sea `HEALTH_OK` antes de continuar.
5. Crea una VM Debian o Ubuntu Server con el disco en el `pool` Ceph. Instala SSH para comprobar conectividad y realiza una migración controlada desde la interfaz de Proxmox.
6. Crea un grupo HA, añade los tres nodos y registra la VM como recurso HA. Simula la indisponibilidad de un nodo según el procedimiento del docente; mide la interrupción mediante `ping` y una sesión SSH desde otra máquina.

9. Autoevaluación

1. ¿Qué es un SPOF?
2. ¿Qué representan MTBF y MTTR?
3. ¿Qué diferencia hay entre RPO y RTO?
4. ¿Por qué RAID no sustituye a una copia de seguridad?
5. ¿Qué comprueba `option httpchk` en HAProxy?
6. ¿Qué función tiene una IP virtual VRRP?
7. ¿Qué diferencia existe entre un balanceador y un proxy inverso?
8. ¿Qué funciones desempeñan Corosync y Pacemaker?

9. ¿Qué riesgo evita el quórum?
10. ¿Qué es *fencing*?
11. ¿Por qué tres nodos son preferibles a dos para el quórum de Proxmox?
12. ¿Qué dependencias se deben comprobar antes de afirmar que una VM está en alta disponibilidad?

11. Recursos

- [HAProxy](#)
- [Keepalived](#)
- [Pacemaker](#)
- [Corosync](#)
- [Proxmox VE](#)
- [Ceph](#)
- [OpenStack](#)
- [WordPress con HAProxy](#)